A Project of Kinship Detection with Dimensionality Reduction and Tree-Based Models

Iia Maharyta
Lviv, Ukraine
Github – luftboud

Vladyslav Danylyshyn
Lviv, Ukraine
Github – D-VLAD1

Arsen Botsko
Lviv, Ukraine
Github – Starezax

***Abstract*—This project studies facial kinship verification, which is the task of determining whether two people are biologically related based on their facial images. Such methods can have real-world applications in areas such as genealogy research, missing person identification, and forensic analysis. The proposed approach combines facial image preprocessing, Principal Component Analysis (PCA), similarity-based feature computation, and a Decision Tree classifier. The method is simple, interpretable, and computationally efficient, and it demonstrates how linear algebra can be applied to a practical computer vision problem.**

***Index Terms*—kinship verification, principal component analysis, feature extraction, similarity measures, decision tree, linear algebra**

I. Introduction

Facial kinship verification is a computer vision task that aims to determine whether two people are biologically related based only on their facial images. Unlike standard face recognition, this problem is more difficult because relatives may share only subtle facial traits rather than identical appearance. In addition, facial images are affected by factors such as pose, lighting, age, and expression, which makes the task even more challenging.

At the same time, kinship verification has possible real-world applications. It can be useful in genealogy research, missing person identification, and forensic investigations, where estimating family similarity from facial images may provide additional supporting information. Because of this, the problem is not only theoretically interesting, but also practically relevant.

Many modern approaches to kinship verification rely on deep learning methods. Although such models can achieve strong results, they usually require large datasets and significant computational resources. In this project, we instead focus on a simpler and more interpretable approach based on classical methods.

The proposed pipeline uses Principal Component Analysis (PCA) to obtain compact representations of facial images and then computes similarity-based features for pairs of faces. These features are used as input for a Decision Tree classifier. The main goal of the project is to show that tools from linear algebra can be used to build a clear, structured, and practically motivated solution to a real classification problem.

II. Related work

Facial kinship verification is a computer vision task that aims to determine whether two people are biologically related based on their facial images. In contrast to standard face recognition, this problem is more difficult because relatives may share only subtle facial traits, while age, gender, lighting, pose, and facial expression can significantly affect appearance [1], [3].

An important contribution to this field is the Families in the Wild (FIW) dataset, which introduced a large-scale benchmark for kinship recognition and verification [1]. It provided real family images with different relationship types and helped establish kinship verification as a separate research problem.

A classical approach relevant to this project is Principal Component Analysis (PCA). The Eigenfaces method showed that face images can be represented in a lower-dimensional subspace while preserving the main variation in the data [2]. PCA remains useful because it is simple, interpretable, and computationally efficient [2], [4].

More recent studies use deep learning methods to learn more complex facial representations for kinship verification [3]. Such methods often achieve better performance, but they usually require larger datasets and more computational resources. In this project, we focus on a simpler and more explainable pipeline based on PCA features.

Decision Tree Classifier is a widely used supervised learning method that builds a model in the form of a tree structure. A clear and accessible explanation of this approach is provided in the Scikit-learn User Guide, which describes how decision trees recursively split the feature space based on feature values in order to maximize class separation. In the context of this project, decision trees are applied to similarity-based feature vectors derived from PCA embeddings. Since these features are numerical and relatively low-dimensional, decision trees can effectively partition the space and learn decision boundaries that distinguish between related and unrelated pairs [5].

Overall, prior work provides both the application context for kinship verification [1], [3] and the theoretical basis for the methods used in this project [2], [4], [5].

III. Theory Behind the Project

A. Mathematical Representation of Facial Images

In this project, each facial image is treated as a mathematical object that can be analyzed using tools from linear algebra.

After preprocessing, every image is converted into a grayscale format and represented as a matrix of pixel intensities.

Let an image have resolution $h \times w$. Then it can be written as a matrix

$$I \in \mathbb{R}^{h \times w},$$

where each element I_{ij} corresponds to the intensity of a pixel.

To apply linear algebra methods, the image matrix is transformed into a vector by flattening:

$$x \in \mathbb{R}^n, \quad \text{where } n = h \cdot w.$$

This operation converts the two-dimensional image into a point in an n-dimensional vector space.

Given a dataset of m images, we construct a data matrix

$$A \in \mathbb{R}^{m \times n},$$

where each row corresponds to a single flattened image:

$$A = \begin{bmatrix} x_1^T \\ x_2^T \\ \vdots \\ x_m^T \end{bmatrix}.$$

Before applying further analysis, the data is mean-centered. This step ensures that each feature has zero mean across the dataset. Let

$$\mu = \frac{1}{m} \sum_{i=1}^{m} x_i$$

be the mean vector. Then each sample is transformed as

$$\tilde{x}_i = x_i - \mu.$$

In matrix form, this corresponds to

$$\tilde{A} = A - \mathbf{1}\mu^T,$$

where $\mathbf{1}$ is a column vector of ones.

Mean-centering is essential because PCA is designed to measure variance around the mean of the dataset. Without centering, the method would be affected by the absolute position of the data in space rather than by the true directions of variation.

This representation allows us to interpret the dataset as a collection of points in a high-dimensional vector space. As a result, operations such as distance computation, projection, and dimensionality reduction can be naturally applied using linear algebra techniques.

B. Principal Component Analysis

At first, we considered using Singular Value Decomposition (SVD), since PCA is closely related to the decomposition

$$A = U\Sigma V^T.$$

However, for this project, the full SVD was not needed. In particular, we did not use the matrix U, because our goal was not to work with the full decomposition itself, but only to find the main directions of variation in the feature space.

For this reason, we used an approach based on the matrix

$$A^T A.$$

The eigenvectors of this matrix correspond to the principal directions used in PCA, so this was enough for constructing the reduced basis and computing embeddings. In this way, the method stays consistent with PCA theory while being more practical for the current implementation. So, in the following subsections, we consider the theoretical formulation of PCA from this point of view.

1) Variance Maximization Viewpoint: The main idea of PCA is to find a direction $w \in \mathbb{R}^n$ such that the projections of the data onto this direction have maximal variance. For a unit vector w, the projection of a centered sample $\tilde{x}_i$ onto w is

$$\tilde{x}_i \cdot w.$$

The variance of these projected values over the dataset is

$$\frac{1}{m} \sum_{i=1}^{m} (\tilde{x}_i \cdot w)^2.$$

In matrix form, this quantity can be written as

$$\frac{1}{m} \|\tilde{A}w\|^2 = \frac{1}{m} w^T \tilde{A}^T \tilde{A} w.$$

Thus, PCA seeks a unit vector w maximizing

$$w^T \tilde{A}^T \tilde{A} w \quad \text{subject to} \quad \|w\| = 1.$$

This is a classical optimization problem from linear algebra, and its solution is given by an eigenvector corresponding to the largest eigenvalue of the matrix

$$\tilde{A}^T \tilde{A}.$$

2) Covariance Structure and Eigenvectors: The matrix

$$C = \frac{1}{m-1} \tilde{A}^T \tilde{A}$$

is the covariance matrix of the centered dataset. It describes how the pixel coordinates vary together across all facial images. Since C is symmetric, it has a complete set of orthogonal eigenvectors and real eigenvalues:

$$Cv_j = \lambda_j v_j, \qquad j = 1, 2, \ldots, n.$$

Each eigenvector v_j defines a direction in the original feature space, and the corresponding eigenvalue λ_j measures how much variance is captured along that direction. The eigenvectors are ordered according to decreasing eigenvalues:

$$\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_n \geq 0.$$

The first principal component is the direction of maximum variance. The second principal component is orthogonal to the first and captures the largest remaining variance, and so on. Therefore, PCA constructs a new orthogonal basis aligned with the intrinsic structure of the data.

3) Dimensionality Reduction: To reduce dimensionality, only the first k eigenvectors are selected, where $k \ll n$. These vectors are assembled into a matrix

$$W_k = \begin{bmatrix} | & | & & | \\ v_1 & v_2 & \cdots & v_k \\ | & | & & | \end{bmatrix} \in \mathbb{R}^{n \times k}.$$

This matrix defines a k-dimensional subspace that captures the most important variation in the dataset. Each centered facial image $\tilde{x}_i$ is then projected onto this subspace:

$$z_i = \tilde{x}_i W_k,$$

where

$$z_i \in \mathbb{R}^k$$

is the reduced representation of the image.

These vectors z_i are the PCA embeddings used later in the project. Instead of storing and comparing the original high-dimensional pixel vectors, we work with compact feature vectors that preserve the dominant facial structure.

4) Geometric Interpretation: From a geometric point of view, PCA performs an orthogonal projection of the data onto a lower-dimensional subspace. This subspace is chosen so that the projected points retain as much variance as possible. In other words, PCA finds the best linear low-dimensional approximation of the dataset in the least-squares sense.

This interpretation is important in the context of linear algebra. PCA is not simply a compression technique; it is a change of basis followed by projection onto a subspace spanned by the leading eigenvectors. The basis vectors are orthogonal, which makes the new coordinates independent in the geometric sense and easier to analyze.

5) Why PCA is Suitable for Facial Images: Facial images are structured data, so the pixels in them are not independent. Neighboring parts of the face usually change together, and general facial properties such as shape, proportions, and placement of features create strong correlations between pixels. Because of this, the true complexity of facial data is much lower than the full number of pixels.

PCA uses this property to find the main directions in which faces vary. These directions capture the most important changes in facial appearance across the dataset. As a result, PCA reduces redundancy and noise while keeping the most informative features.

That is why PCA is a suitable method for this project. It gives a simple and mathematically clear way to transform facial images into compact feature vectors, which can then be used for similarity computation between image pairs.

C. Feature Computation

After obtaining PCA embeddings, the next step is to construct features that describe the relationship between two facial representations. Since the task of the project is kinship verification, the classifier does not work with a single embedding, but with a pair of embeddings corresponding to two people.

Let

$$z^{(1)}, z^{(2)} \in \mathbb{R}^k$$

be the PCA embeddings of two face images. From this pair, a compact similarity feature vector is computed.

First, the absolute difference vector is defined as

$$\delta = \left| z^{(1)} - z^{(2)} \right|.$$

Based on these embeddings, several numerical features are extracted.

The Euclidean distance is computed as

$$d_E\left(z^{(1)}, z^{(2)}\right) = \left\| z^{(1)} - z^{(2)} \right\|_2 = \sqrt{\sum_{i=1}^{k} \left(z_i^{(1)} - z_i^{(2)} \right)^2}.$$

It measures the geometric distance between the two vectors in the reduced PCA space.

The Manhattan distance is given by

$$d_M\left(z^{(1)}, z^{(2)}\right) = \sum_{i=1}^{k} \left| z_i^{(1)} - z_i^{(2)} \right|.$$

This feature captures the total absolute difference across all coordinates.

The cosine similarity is computed as

$$\cos(\theta) = \frac{z^{(1)} \cdot z^{(2)}}{\|z^{(1)}\| \, \|z^{(2)}\|}.$$

This value reflects how similar the directions of the two embeddings are, independently of their magnitudes. In implementation, if one of the vectors has zero norm, cosine similarity is set to 0 in order to avoid division by zero.

Several statistical characteristics of the absolute difference vector are also used. Its mean value is

$$\mu_\delta = \frac{1}{k} \sum_{i=1}^{k} \delta_i,$$

the maximum component is

$$\delta_{\max} = \max_{1 \le i \le k} \delta_i,$$

and the standard deviation is

$$\sigma_\delta = \sqrt{\frac{1}{k} \sum_{i=1}^{k} (\delta_i - \mu_\delta)^2}.$$

In addition, two more algebraic similarity features are included. The dot product of the embeddings is

$$z^{(1)} \cdot z^{(2)} = \sum_{i=1}^{k} z_i^{(1)} z_i^{(2)},$$

which reflects their overall alignment in the feature space.

The squared Euclidean distance is also computed:

$$d_E^2\left(z^{(1)}, z^{(2)}\right) = \sum_{i=1}^{k} \left(z_i^{(1)} - z_i^{(2)} \right)^2.$$

Unlike the standard Euclidean distance, this feature avoids the square root operation and preserves the squared scale of the coordinate differences.

As a result, each pair of embeddings is represented by the following feature vector:

$$\left[d_E,\ d_M,\ \cos(\theta),\ \mu_\delta,\ \delta_{\max},\ \sigma_\delta,\ z^{(1)} \cdot z^{(2)},\ d_E^2\right].$$

Thus, instead of feeding the classifier with raw PCA embeddings directly, the model receives a compact and interpretable vector of similarity-based features that summarizes the relation between two faces.

D. Decision Tree Classifier

The Decision Tree classifier is used in this project as the final prediction step. It does not work directly with raw images. Instead, it takes as input the feature vectors computed from pairs of PCA embeddings.

From the point of view of linear algebra, this means that each pair of facial images is first transformed into a vector in a feature space:

$$f \in \mathbb{R}^d,$$

where d is the number of similarity features.

Thus, the role of linear algebra in this stage is to provide a meaningful numerical representation of each image pair. The classifier then uses these vectors to separate the data into two classes: kin and non-kin.

So, even though the Decision Tree itself is not a linear algebra method in the same sense as PCA, it still depends on the vector-space representation built in the previous stages of the project.

IV. Project Pipeline

A. Short Overview

The project pipeline transforms raw facial images into features suitable for classification. First, images are preprocessed and converted into a consistent vector representation. Then PCA is applied to obtain compact embeddings. Based on these embeddings, similarity features are computed for pairs of images, which are finally used as input for the classifier.

B. Data overview

The project uses the Families in the Wild (FIW) dataset [6], which is a large-scale benchmark for facial kinship recognition. According to the original dataset paper, it contains over 10,000 family photos from 1,000 families.

A key advantage of FIW is that it reflects realistic variability in facial appearance. The images differ in pose, lighting, age, expression, and image quality, which makes the task more difficult but also more representative of real applications. Because of this, FIW is widely used as a benchmark for evaluating kinship verification methods.

In this project, FIW is used as the source of facial images for preprocessing, PCA-based feature extraction, and pairwise similarity computation. Since the dataset provides labeled family relationships, it is suitable for constructing image pairs and later training a classifier to distinguish kin from non-kin pairs.

C. Preprocessing Flow

This stage applies in practice the steps described above and prepares the images for PCA.

First, all images are loaded from the dataset. Each image is converted to grayscale, since color is not needed for our task and intensity values are enough to describe facial structure.

After that, each face is resized to a fixed resolution of 50×50 pixels. This is necessary so that all images have the same size and can later be represented in the same vector space.

Then the pixel values are normalized to the range $[0, 1]$, which makes the data more numerically stable.

Once the image is processed, it is flattened into a one-dimensional vector. These vectors are collected into a matrix, where each row represents one image.

Finally, mean-centering is applied using the global mean vector of the dataset. This gives the centered matrix that is used in the PCA step.

As a result, the preprocessing stage converts raw facial images into a consistent matrix form that can be used for further analysis and feature extraction.

D. Feature Extraction via Principal Component Analysis

At this stage, PCA is applied in practice to the centered data matrix obtained after preprocessing.

The input is a matrix A, where each row corresponds to one centered facial image represented as a vector. Based on the theoretical formulation, the implementation computes the matrix

$$A^T A,$$

which captures the relationships between features and is used to find the principal directions.

Next, eigenvalue decomposition is performed using `np.linalg.eigh`, since the matrix is symmetric. This step returns eigenvalues and corresponding eigenvectors.

The eigenvalues are then sorted in descending order, and the eigenvectors are reordered accordingly. Only the first `max_features_amount` eigenvectors are selected. These vectors form the reduced basis used in further computations.

Finally, each centered image vector is projected onto this basis:

$$z = xW_k,$$

where W_k contains the selected eigenvectors. As a result, each image is represented by a lower-dimensional vector.

These vectors (embeddings) are used as the final representation of facial images and serve as input for the next stage, where similarity features between pairs of images are computed.

E. Similarity Feature Computation

In the implementation, similarity features are computed directly from pairs of PCA embeddings.

Each embedding is first converted into a one-dimensional numeric array. For a pair of embeddings $z^{(1)}$ and $z^{(2)}$, their element-wise absolute difference

$$\delta = \left| z^{(1)} - z^{(2)} \right|$$

is calculated. This difference vector is then used to obtain the mean, maximum value, and standard deviation of component-wise differences.

At the same time, the Euclidean distance, Manhattan distance, cosine similarity, dot product, and squared Euclidean distance are computed. The cosine similarity requires division by the product of vector norms, so in the special case where this denominator is zero, the similarity is assigned the value 0 for numerical stability.

For every pair of images, these eight values are combined into one feature vector. After that, all feature vectors are stacked together into a feature matrix:

$$X \in \mathbb{R}^{m \times 8},$$

where m is the number of image pairs, and each row corresponds to one pair.

This matrix is then used as the input to the classification model. In this way, the similarity computation stage transforms pairs of PCA embeddings into a structured dataset suitable for supervised learning.

F. Decision Tree Classification

After computing similarity-based features for each pair of embeddings, the final step of the pipeline is classification. At this stage, each pair of images is represented by a fixed-length feature vector, and the task is to predict whether this pair belongs to the class of biologically related people or not.

For this purpose, we use a Decision Tree Classifier. The input to the model is the feature matrix

$$X \in \mathbb{R}^{m \times 8},$$

where each row corresponds to one pair of faces, and each column corresponds to one similarity feature. The target vector

$$y \in \{0, 1\}^m$$

contains binary labels: 1 for kin and 0 for non-kin.

In the implementation, the feature matrix for training is constructed from the training pairs using the function `build_pair_feature_matrix`. After that, a decision tree model is created with the following parameters:

- `max_depth = 12`,
- `class_weight = balanced`,
- `random_state = 42`.

The depth limit is introduced to prevent the tree from becoming too complex and overfitting the training data. The `balanced` class weighting is used so that the classifier takes both classes into account more evenly, even if the dataset is not perfectly balanced. Fixing the random seed ensures reproducibility of the results.

The model is then trained on the feature matrix X_{train} and the corresponding label vector y_{train}. After fitting, predictions on the training set are computed in order to estimate how well the tree has adapted to the training data. In our experiment, the training accuracy was

$$0.5864.$$

To evaluate the model, the same feature extraction procedure is applied to the test pairs, producing the matrix X_{test}. The trained classifier predicts labels for these pairs, and the results are evaluated using accuracy, precision, recall, F1-score, and the confusion matrix.

The obtained test accuracy is

$$0.5303.$$

The classification report shows that the model performs similarly on both classes, but the overall predictive quality remains rather limited. For class 0 (non-kin), precision is 0.52, recall is 0.58, and F1-score is 0.55. For class 1 (kin), precision is 0.54, recall is 0.48, and F1-score is 0.51.

The confusion matrix is

$$\begin{bmatrix} 27949 & 20398 \\ 25786 & 24195 \end{bmatrix}.$$

This means that the classifier correctly predicted 27949 non-kin pairs and 24195 kin pairs, while making a substantial number of errors in both directions. In particular, the model misses many true kin pairs, which is reflected in the lower recall for class 1.

Overall, the Decision Tree serves as a simple and interpretable baseline classifier in the pipeline. It allows us to test whether the extracted similarity features contain enough information for kinship verification. At the same time, the modest test accuracy indicates that this feature representation and model configuration are not sufficient for high-quality classification, so further improvements are needed.

V. Plan of Future Work

The experimental results show that the current pipeline provides only modest performance, with a test accuracy of approximately 0.53. This indicates that the extracted features and the chosen classifier are not sufficient to reliably distinguish between kin and non-kin pairs. Therefore, future work should focus on improving the main components of the pipeline in a systematic way.

- **Improvement of PCA representation.** Although PCA provides a compact embedding, it may discard information that is important for kinship verification. Future work should include systematic experiments with different numbers of principal components k, as well as an analysis of explained variance. This would help determine how much information is preserved and whether the chosen dimensionality is appropriate for the task.
- **Improvement of feature representation.** The current feature set is based on simple similarity measures, which may not capture subtle facial relationships. Future work should explore richer pair representations, such as element-wise products, concatenation of embeddings, or other transformations that preserve more information about the relation between two faces. This is important because the classifier can only use the information encoded in these features.

- **Improvement of the classification model.** In this project, the Decision Tree serves as a simple and interpretable baseline. However, the obtained results suggest that its capacity is limited for this task. Future work should therefore include more expressive models, such as Random Forests or other ensemble methods, which may better capture nonlinear relationships in the feature space.
- **Hyperparameter tuning.** The current implementation uses fixed parameters for PCA and for the Decision Tree classifier. A more systematic search over parameters, such as the number of PCA components, tree depth, and splitting criteria, should be carried out in order to better understand their effect on the final results.
- **Error analysis.** The confusion matrix shows that the model makes a substantial number of errors in both classes, including many missed true kin pairs. Future work should include a closer analysis of misclassified examples in order to determine whether the main source of error lies in the data quality, the feature design, or the classifier itself.
- **Visualization and interpretability.** Additional visual analysis could make the behavior of the model easier to understand. For example, it would be useful to visualize the PCA space, inspect the distribution of kin and non-kin pairs, and analyze feature importance in the classification stage.

Overall, the main direction of future work is to move from a simple baseline pipeline toward a more expressive representation of facial similarity and a stronger classification model, while at the same time ensuring a correct and reliable evaluation setup.

VI. Repository

The code for this project is available in the corresponding GitHub repository. It includes the full implementation of the pipeline, from preprocessing and PCA to feature computation and classification. The repository can be used to reproduce the results and extend the current approach.

References

[1] Robinson, J. P., et al. "Families in the Wild: Large-Scale Kinship Image Database and Benchmarks." 2016.
[2] Turk, M., and Pentland, A. Eigenfaces for Recognition. 1991.
[3] Dahan, E., and Keller, Y. A Unified Approach to Kinship Verification. 2020.
[4] Shlens, J. A Tutorial on Principal Component Analysis.
[5] Scikit-learn Developers. Decision Trees — Scikit-learn Documentation.
[6] Kaggle Dataset: Northeastern SMILE Lab - Recognizing Faces in the Wild